

Trae + Spec Coding + Figma MCP 开发实践

本文作者：包卓娜，TRAE技术专家

一页速览

本文档复盘了利用 Trae、Spec Coding 与 Figma MCP 将“2025 年度报告”从国内版迁移适配至国际版的工程实践。核心目标是探索如何借助 AI 工具链，高效地基于新的 PRD 和 Figma 设计稿完成代码的差异化修改。



核心方法与路线

- 环境准备：**配置 Lark、BAM 接口及 Figma Dev Mode 的 MCP，为 AI 工具链提供必要的上下文读取能力。
- 需求拆解 (Spec Coding)：**使用 `specify-cn-cli` 读取 PRD 文档，自动生成开发规范、执行计划与任务列表。实践证明，AI 生成的初版任务清单需人工审核和调整，以确保其准确对齐“基于现有代码做 diff 修改”的核心目标，而非从零开发。
- 设计稿还原 (Figma to Code)：**探索了三种将 Figma 设计稿落地为代码的路径，最终验证了“截图 + 自然语言指令”的组合在当前场景下效果最佳。

关键结论与建议

- Spec Coding 较为成熟：**
 - 能够准确理解 PRD 并生成详细、可执行的任务列表。但仍需人工介入，校准 AI 对核心需求的理解（例如，是“迁移修改”还是“全新实现”）。
 - 操作流程有点复杂，如需求不算特别复杂，直接让 TRAE 拆任务和写代码就足够。
 - 建议配合 `specify-cn-cli` 的场景：
 - 大型或跨多人协作需求，希望先有一份清晰的 PRD → Spec → 任务清单，再按步骤执行。
 - 对“任务粒度可控、可复盘”和“需求-实现强对齐”要求比较高，希望借助 Spec 文档来约束 AI 的实现方式。
- Figma to Code 挑战较大：**
 - 直接使用 Figma Section URL：**本次场景无论是直接修改代码还是先生成 Plan，AI 对布局、样式和逻辑的理解均存在较大偏差，产出代码几乎不可用。

- **推荐采用截图输入：**向 AI 提供页面截图和明确的自然语言指令，在还原文案、逻辑和图片资源方面效果最好。此方法的主要局限在于无法精确还原字体、间距等视觉样式细节。
- **AI 交互建议：**对于复杂任务，建议采用“分步执行、小步快跑”的策略。例如，先让 AI 完成第一阶段任务，验证通过后再继续，而不是一次性执行所有任务。
- **初步探索：**本次整体开发效率不提反降，预计开发1.5d，跟AI搏斗后2.5d开发完成😄

背景与目标

本次实践的核心任务是基于已有的“Trae 2025 国内年度报告”代码，利用 Trae、`specify-cn-cli` 及 Figma 等 AI 辅助工具，根据新的国际版 PRD 与 Figma 设计稿进行差异化修改，最终交付国际版年度报告。

开发环境配置

为了让 AI 工具能够读取 PRD、BAM 接口文档和 Figma 设计稿，需要预先配置好相关的 MCP (Meta-protocol Command Platform)。

1. 读取飞书文档 (PRD)

为读取飞书 wiki 中的 PRD，我们配置了 Lark MCP。

- **权限申请：**使用飞书团队提供的 `coding assistant` 应用权限，无需自行申请独立 App。
- **MCP 配置：**在 MCP 设置中选择“手动配置导入”，并使用以下 JSON。

代码块

```
1  {
2    "mcpServers": {
3      "LarkDocs": {
4        "command": "npx",
5        "args": [
6          "-y",
7          "--registry",
8          "http://bnpm.byted.org/",
9          "@byted/mcp-lark-docs@latest"
10       ],
11       "env": {
12         "AUTH_CALLBACK_PORT": "6688",
13         "AUTH_CALLBACK_ENDPOINT": "/lark/callback",
14         "LARK_TENANT": "bytedance",
15         "LARK_APP_ID": "cli_a9a96fe740b89bc2",
16         "LARK_APP_SECRET": "GcfWdzMJTCya13Y7coBKNFkQKQnBPJWv"
17       }
18     }
19   }
```

```
19   }
20   }
21
```

2. 读取 BAM 接口文档

配置 BAM API Doc MCP，此项无需额外设置，默认集成。

B

BAM API DOC

Local

提示

该 Server 提供与 BAM API 相关的各项工具，支持查询 BAM 接口定义、查询单个 PSM 的接口...

⚙️

3. 读取 Figma 设计稿

为实现设计稿的读取与代码转换，需要配置 Figma Dev Mode MCP。

核心流程：Spec Coding 实践

我们利用 `specify-cn-cli` 工具，通过读取 PRD 实现了从需求到代码任务的自动化拆解。整个过程分为读取需求、创建规范、生成计划与任务，并最终通过人工审核进行调整。

前置条件：已安装 `specify-cn-cli`

1. 读取与解析需求文档

首先，指令 AI 读取指定的飞书 PRD 文档，并将其转换为本地 Markdown 文件。

代码块

- 1 读取 `https://bytedance.larkoffice.com/wiki/QLHsw5BlxiYswjk83TfcpwQdnPd`，将其转化为md文档，按照`./ai-coding/需求名/需求名.md`方式存储
- 2

PRD_国际版年度报告.mdPreview PRD_国际版年度报告.md ×

报告内容		
- 需要加入 twitter 分享、格子日历 - 删除部分国内数据，新增部分海外数据维度：最常用 agent、采纳次数、ai 活跃日期 - 视觉部分需要加入生日帽 - 待讨论-是否需要支持中英双语?		
类别	数据维度	Global 文案 (EN)
报告首页 ✓	/	Connecting... 正在连接... Server: TRAE.ai 服务器: TRAE.ai Syncing your 2025 recap... 正在同步您的 2025 年总结... Identity: The Real AI Engineer 身份: 真正的 AI 工程师 Status: Loaded. 状态: 已加载。
报告首页-隐私协议 ✓	/	This recap is generated from your TRAE activity and is visible only to you, unless you choose to share it. 本总结是根据您的 TRAE 活动生成的，仅对您可见，除非您选择分享。
	用户总数	Hey {username}, 嘿 {username}, 欢迎回来。 In 2025, TRAE was built with a growing community of builders worldwide. 今年, TRAE 是由全球不断壮大的建设者社区共同打造的。

在 AI 完成初步转换后，可以对其进行追问，补充或修正关键信息。例如，我们补充了数据采集时间、新用户判断标准等细节。

代码块

1

修改需求文档并优化，需求的核心内容是报告内容和埋点需求

2

修改点：

3

1. 数据采集的开始时间 utc+0时区的2026.01.01的0点0分0秒

4

2. 新用户判断，注册时间在utc+0时区的2026.01.01的0点0分0秒之前

5

2. 创建规范、计划与任务

基于处理后的需求文档，依次执行指令生成开发规范、执行计划和任务列表。

代码块

1

创建规范

2

#speckit.specify.md

3

需求文档为.ai-coding/PRD_国际版年度报告/PRD_国际版年度报告.md

4

5

创建执行计划

```
6 .claude/commands/speckit.plan.md
7
8 # 创建任务
9 .claude/commands/speckit.tasks.md
10
```

3. 任务评审与人工调优

AI 初版任务清单存在一个核心偏差：它将需求理解为“从零开始实现所有功能”，而我们的目标是“基于国内版代码进行差异化修改”。这导致初版任务列表包含了大量重复性实现工作。

因此，我们向 AI 提供了明确的调整指令，以校准其理解：

代码块

```
1 需要对任务列表进行调整：
2 1. 你目前生成的任务中，只迁移国内版通用 `assets` 到国际版目录，其他功能都是自行实现。我期望迁移国内版所有内容，在国内版基础上进行国际版的diff修改，而不是都重新实现。
3 2. 支持 Twitter 分享，检索项目中已有能力进行复用，不用自行实现。
4 3. 年度角色与总结页的日历热力图，检索项目中已有能力进行复用，只修改样式，由横向展示改为纵向展示。
5 4. 接口补充，...报告结构使用trae_2025_i18n_report字段。
6 5. figma,使用mcp读取设计稿，对比跟国内版diff进行修改 ...
7 6. 无需单独添加“生日帽”等新视觉元素，遵循还原figma设计稿即可。
8 7. 在 `api.ts` 中适配新接口字段...并且接口改为使用bam方式使用...
9
```

经过调整，最终生成的任务列表更贴合实际需求，明确了迁移、复用和差异化修改的策略。

4. 任务执行与效果评估

在执行 `speckit.implement.md` 时，我们采用了分阶段执行的策略，即先完成并验证 `phase 1` 的效果，再继续后续阶段。



Spec Coding 实践效果

- **优点：**AI 对 PRD 的逻辑理解基本准确，能够生成详细且结构化的任务，使得项目能够快速启动和运行。
- **缺点：**当输入信息较为模糊或存在歧义时（例如，直接提供多个 Figma Section URL），AI 的执行效果会大打折扣，可能只完成少量修改。此外，PRD 中的图片信息在任务转化过程中会丢失，影响了需求的完整性。

下表记录了各阶段任务的完成情况与效果评估：

阶段任务	核心子任务	完成度	效果分
阶段 1: 迁移与设置	- 将国内版代码完整复制到国际版目录 - 修复导入路径与依赖问题 - 配置新路由入口 - 适配新接口字段及调用方式 (BAM SDK)	完成	100
阶段 2: 国际化适配 (Diff 修改)	- 替换所有中文文案为英文 - 调整日期格式 - 移除国内版特有组件 - 适配角色计算逻辑与文案	基本完成	90
阶段 3: 功能与样式调整	- 视觉还原：使用 MCP 读取 Figma 设计稿 - 日历热力图：修改组件为纵向展示，并对接新数据源 - Twitter 分享：复用项目中现有分享组件	未完成	0

阶段 3 失败原因分析：在读取 Figma 设计稿环节，我们一次性提供了包含多个页面的 Figma Section 链接，AI 仅识别出两行文案差异，完全无法满足视觉还原的需求，导致后续任务失败。

核心流程：Figma 设计稿还原探索

在 `Spec Coding` 解决了代码框架和业务逻辑迁移的问题后，我们面临的核心挑战是如何精确地将 Figma 设计稿中的视觉样式和交互细节还原到代码中。我们探索了三种不同的方法。

尝试 1：Figma Section + Trae 直接修改

此方法最为直接，即向 Trae 提供单个页面的 Figma Section 链接，并指令其直接修改对应的代码文件。

- 输入指令示例：**

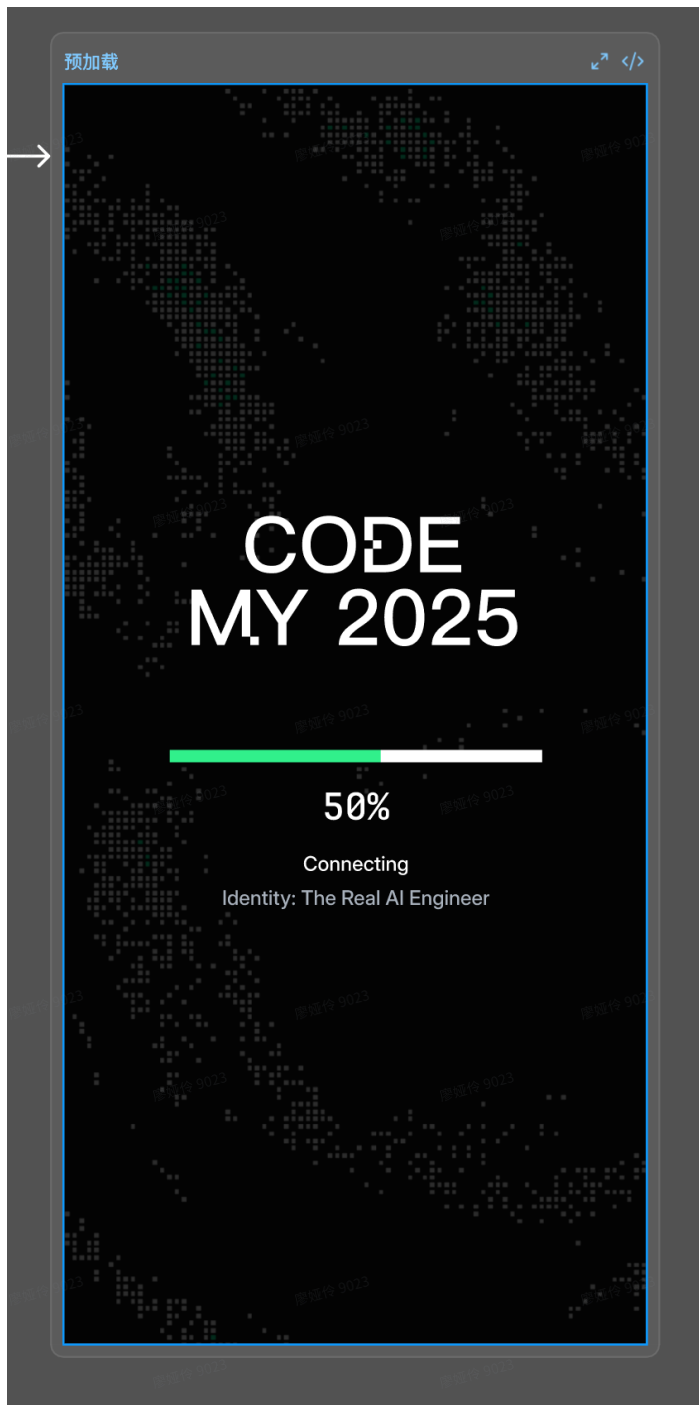
代码块

```
1 [figma section链接](https://www.figma.com/design/...) 读取figma设计，100%还原设计稿，修改apps/website-i18n/src/routes/events/annual-2025/components/LoadingPage.tsx
2
```

- 实际效果：非常差。**
 - AI 错误地修改了组件的布局和加载逻辑，导致页面无法正常运行。
 - 错误识别出需要替换背景图，未正确应用。

主要差异仅为文案和字体，但 AI 的修改却破坏了原有结构。

预期设计稿

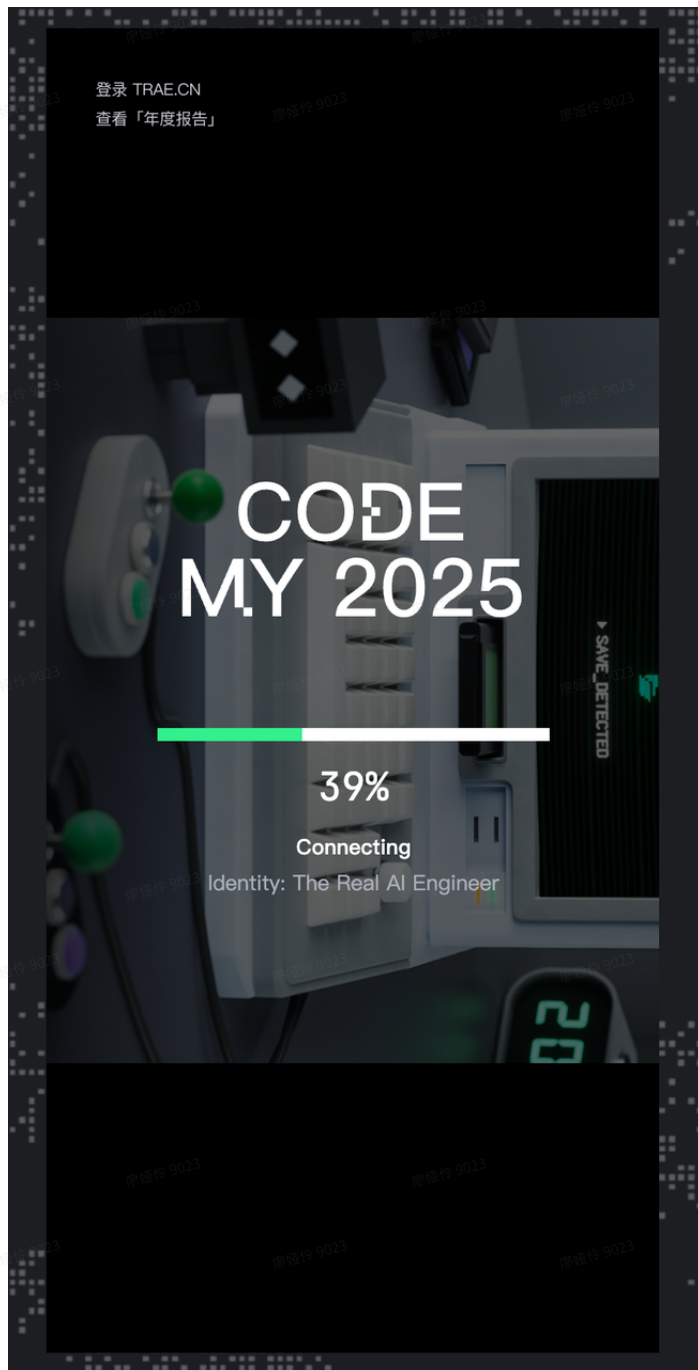


AI 修改计划

国内版



最终效果



尝试 2：Figma Section + 先输出 Plan 再执行

为了增加可控性，我们调整了交互方式，要求 Trae 在读取 Figma Section 后先输出修改计划（Plan），待人工审核确认后再执行。

- 交互流程：

- a. 提供 Figma Section 链接。
- b. Trae 输出修改计划。
- c. 人工审查并调整 Plan（例如，明确指出图片资源位置和文案修改要求）。
- d. 执行调整后的 Plan。

- 实际效果：非常差。

- 即使在人工干预 Plan 的情况下，AI 依然无法正确识别 Figma 中的关键信息（如文案、图片资源）。
- 我们尝试了多种模型（ Gemini-3-Pro-Preview 、 c-test 等），结果均不理想。

尝试 3：Figma 截图 + 自然语言指令

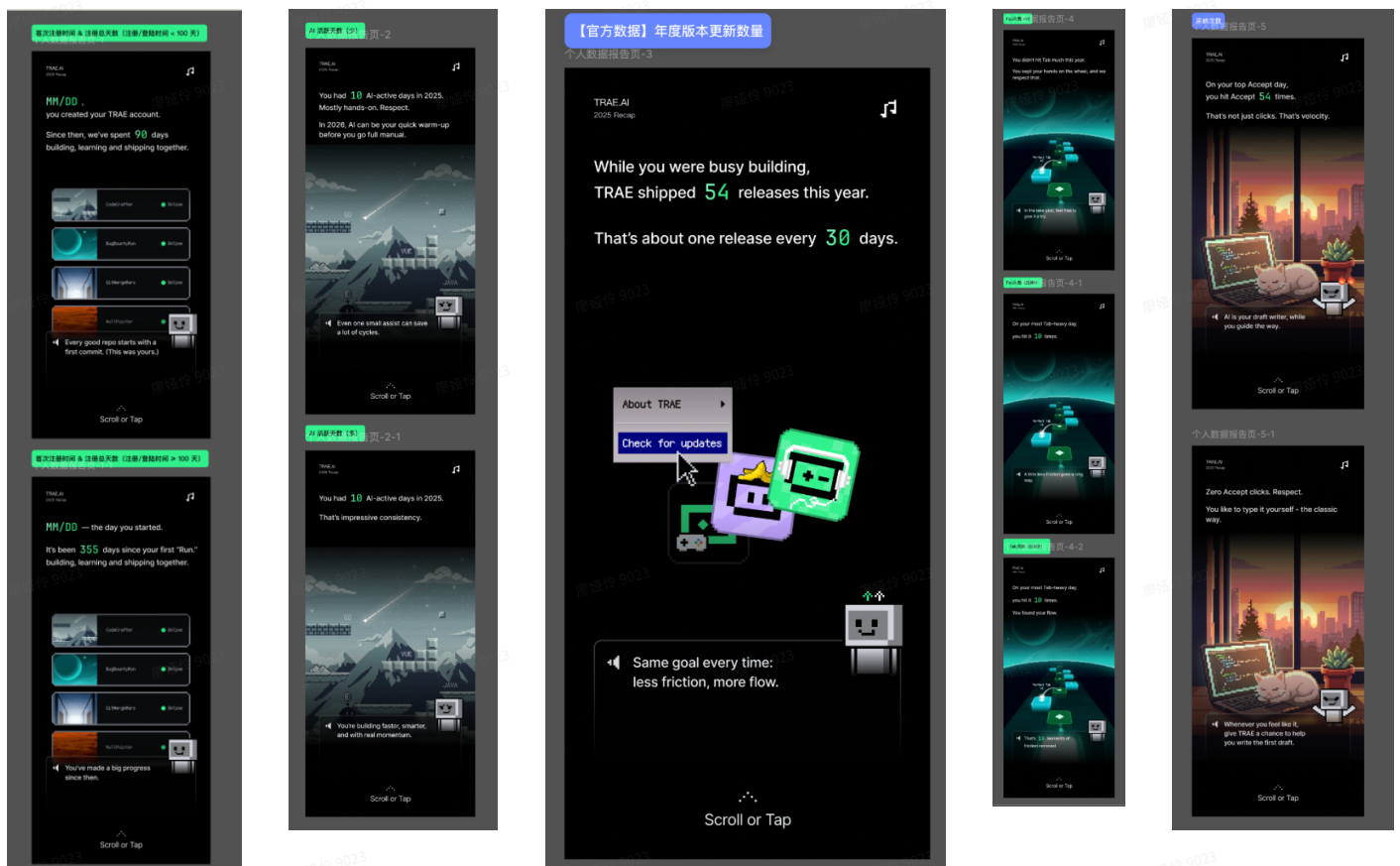
鉴于直接读取 Figma 设计稿的路径受阻，我们尝试了一种更“原始”但有效的方法：使用截图作为视觉输入。

输入指令示例：

代码块

- 图片分别是DataPersonOnePage到DataPersonFivePage的截图，有的页面有多种case，还原设计稿里的文案，把对应的图片资源，根据命名正确引入到页面中...
-

输入材料：提供 1-5 张页面截图，覆盖不同的 UI 状态和场景。



实际效果：基本可用，效果最佳。

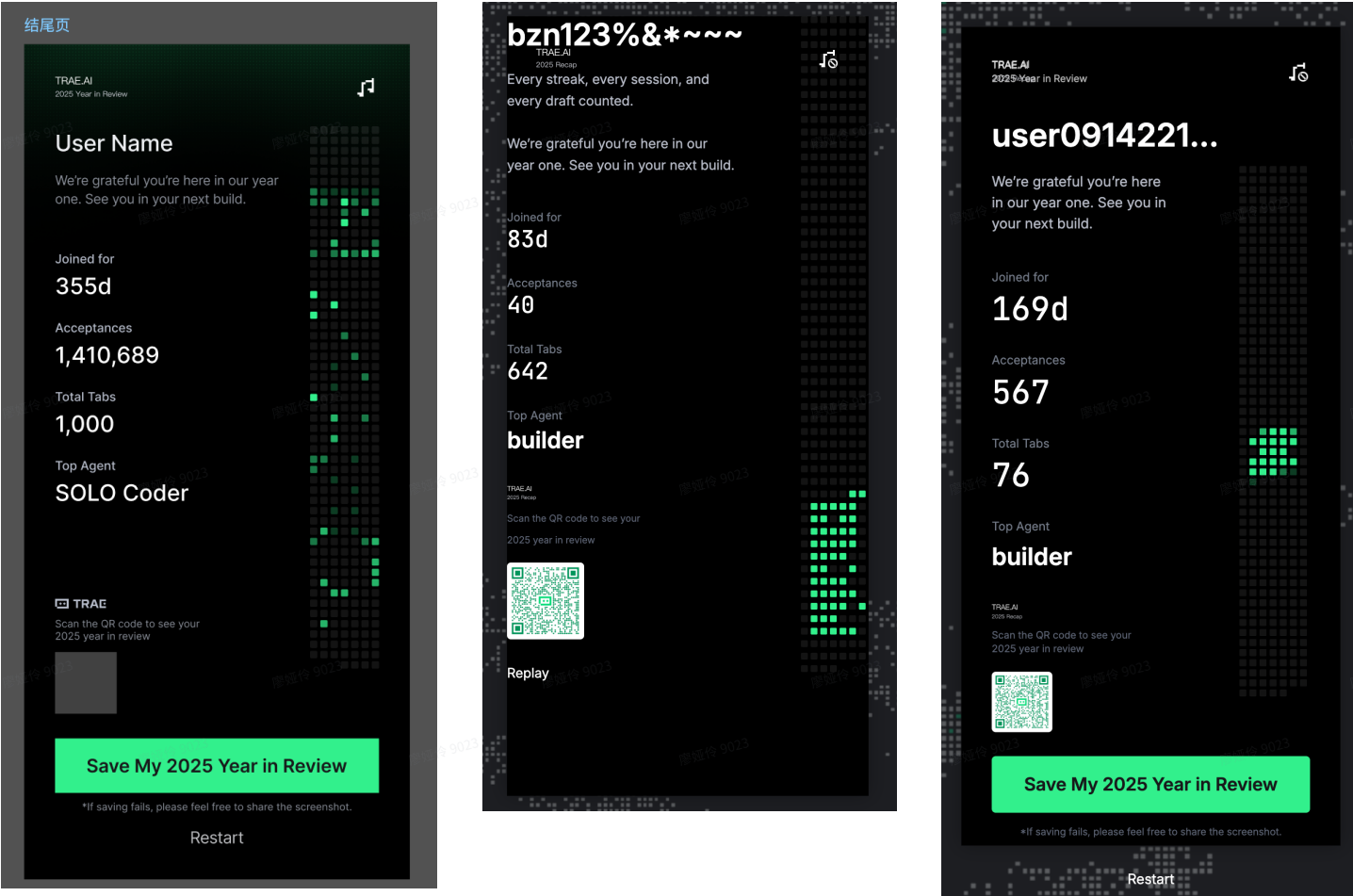
- ✅ AI 能够准确识别并修改代码中的文案和多 case 逻辑。
- ✅ 能够根据命名约定，正确引入和替换图片资源。
- ⚠️ 首次运行时可能产生构建错误，需要追问让 AI 自行修复（ 执行 npm run build 验证修改是否有报错并修复 ）。

- ✗ 无法读取截图中的精确样式信息，如字体、颜色、间距等。
- ✗ 对于一个页面包含多种 case 的复杂情况，可能无法完全覆盖所有场景（例如，3 种 case 只实现了 2 种）。

尝试 4：Figma Section + Figma 截图 + 自然语言指令

生成一个新内容较多的页面，也难以令人满意

设计稿 给出figma section + 截图 多轮自然语言指令调整



方法对比与总结

为了更清晰地评估这三种方法，我们汇总了其对比情况如下：

方法	输入方式	优点	主要问题	推荐度
尝试 1	Figma Section URL	理论上最直接，无需手动处理	- 严重错误理解布局和逻辑 - diff修改破坏原有代码结构 - 读的变量部分是错的	不推荐

尝试 2	Figma Section URL + 人工审核 Plan	增加了中间审查环节，更可控	- 无法从 Figma 准确提取信息 - 即使干预 Plan，效果依然很差	不推荐
尝试 3	页面截图 + 自然语言指令	- 文案和逻辑还原度高 - 能基本正确处理图片资源	- 无法还原精确样式 (字体、间距等) - 复杂页面可能遗漏部分 case - 结果不稳定，可能需重试	相对推荐

更多信息请关注TRAEE企业版服务号👉





微信搜一搜



TRAEE企业版